Lab 3: Architectural Pattern Selection - Justification

Name: N S Raj Nanjappa | SRN: PES1UG24CS287 | Section: E | PES University, Dept. of CSE

Architecture Selection

We chose Layered Architecture for the OpenAPI Mock Server Generator System.

It was compared with Microservices (independent scaling and fault isolation, but network latency and high operational complexity) and Client-Server (simple, centralised, but a single point of failure and a scalability bottleneck). The system has five main components in three layers: Presentation (Management UI / CLI), Business (Mock Server Engine, OpenAPI Spec Parser, Response Generator, Schema Validator, Latency Simulator) and Data (Spec & Config Store). They communicate through the interfaces IMockControl, ISpecParse, IGenerate, IValidate, ILatency and ISpecStore.

Reasons for the Choice

1. The workload is a fixed processing pipeline. Every mock request follows the same ordered steps: route, validate against the schema, generate the JSON payload, then apply latency. Layers with clearly separated responsibilities map directly to this flow, so each concern (parsing, validation, generation, latency) can be developed and tested separately and a component can be replaced if it keeps the same interface.

2. It is a single developer tool that runs locally. The tool is used by one API Developer or QA Engineer on a machine or CI runner and needs no independent scaling per service. Layered keeps deployment to one process and avoids the complexity of service discovery, network calls and distributed data consistency that microservices would add.

Security Advantage

All external input enters through the Presentation layer and the Engine's single HTTP port. Uploaded specs and incoming requests must pass the Spec Parser and Schema Validator in the Business layer before reaching the Data layer, so the Spec & Config Store is never directly exposed. Checks such as the 5 MB spec limit, blocking remote `$ref` URLs (SSRF) and default localhost-only binding (NFR-002) are enforced at one choke point instead of in many services.

Performance Benefit

Components in the Business layer call each other in-process, so a request passes through validation, generation and latency simulation with no network hops. This keeps the overhead small and predictable, which is required to sustain 1,000 requests per second with simulated latency accurate to ±10 ms (NFR-001). A microservices design would add network delay per call and make the latency timing less accurate.

Trade-off Accepted

Layers cannot be scaled independently and tight coupling between layers is possible; this is acceptable here because the interfaces are kept narrow and the load is a single process handling mock traffic.